314581030 徐煜宸 HW3 report

AI prompt

- 這份作業的環境、RandomAgent、plot、main 大部分架構都是助教提供的，我主要補上 `LAAgent.py` 和 `RLAgent.py`。
- LA 的部分我先照上課講義的 Learning Automata 公式去寫，後面再跟 AI 討論要怎麼把 reward 轉成 continuous reward response，也就是現在 code 裡面的 `beta`。
- RL 的部分我有請 AI 幫我檢查我寫的 DQN 架構，包含 replay buffer、target network、epsilon-greedy、以及為什麼這題的 state 要用上一回合雙方 action。
- 報告撰寫的部分，我有請 AI 根據 HW3 PDF、目前 `src/` 內容、以及我跑出來的圖，幫我整理 code explanation 和 free response，但文字內容我有重新調整成我比較看得懂的版本。

大致的 prompt 是：

```
請根據 HW3 PDF 和 314581030_HW3/src 的內容，幫我 review LAAgent 和 RLAgent，
解釋 RLAgent 的概念與流程，並且整理成符合助教要求的 report。
report 需要包含 source code explanation、result analysis、free response questions、
以及 AI prompt description。
```

Code Explain

`PrisonersDilemmaEnv.py`

這個檔案是助教給的 Prisoner's Dilemma environment。每一個 episode 有 100 個 stage，每個 stage protagonist 和 opponent 各選一個 action。

- `0` 代表 Silent，也就是合作。
- `1` 代表 Betrayal，也就是背叛。
- state 是 `[protagonist 上一回合 action, opponent 上一回合 action]`。
- 第一回合還沒有歷史，所以初始 state 是 `[-1, -1]`。

payoff matrix 是題目給的這張：

```
self.payoff_matrix = np.array([
    [(10, 10), (4,  12)],
    [(12, 4),  (6,  6)]
])
```

意思是如果 protagonist 選 Betray、opponent 選 Silent，protagonist 就會拿到 12 分；如果兩邊都 Betray，protagonist 拿到 6 分。

opponent 有四種固定策略：

- `rand`：隨機選 Silent 或 Betray。

- `silent`：永遠 Silent。
- `betray`：永遠 Betray。
- `tft`：Tit for Tat，第一步 Silent，後面模仿 protagonist 上一回合的 action。

這份檔案我只有小修 observation 的型別，讓 `observation_space` 宣告的 dtype 和實際回傳的 state 一致，避免 Gymnasium checker 對 dtype 不一致有意見。遊戲規則、reward matrix、opponent strategy 都沒有改。

`main.py`

`main.py` 是整個實驗的入口。它會對四種 opponent 各跑一次，流程大概是：

```
OPPONENT = ["tft", "rand", "silent", "betray"]
TRAIN_EPISODES = 400
EVAL_EPISODES = 100
```

每個 opponent 都會先建立 environment，然後建立三個 protagonist：

```
agent_rand = RandomAgent(env, total_episodes=TRAIN_EPISODES)
agent_rl = RLAgent(env, total_episodes=TRAIN_EPISODES, seed=fix_seed)
agent_la = LAAgent(env, total_episodes=TRAIN_EPISODES, rho=0.05)
```

真正互動的部分在 `run_simulation()`。每一步 agent 先根據目前 observation 選 action，environment 回傳 reward 和下一個 observation，如果是 training 就呼叫 `agent.update()`，evaluation 則只看目前 policy 表現。

```
action = agent.act(obs, explore=not is_eval)
next_obs, reward, terminate, truncate, _ = env.step(action)

if not is_eval:
    agent.update(obs, action, reward, next_obs, done)
```

所以三個 agent 的差別其實就只在 `act()` 怎麼選 action，以及 `update()` 怎麼學。

`LAAgent.py`

LAAgent 的想法相對單純，它不看 state，只記一個數字 `p_C`，代表下一步選 Silent 的機率。

```
self.p_C = initial_p_c
```

選 action 的時候就是丟一個隨機數：

```
if np.random.rand() < self.p_C:
    return 0
else:
    return 1
```

更新時我用的是 continuous 版本，所以先把 reward 壓到 0 到 1 之間：

```
r_min, r_max = self.env.reward_range
beta = (reward - r_min) / (r_max - r_min)
beta = np.clip(beta, 0.0, 1.0)
```

直覺上，`beta` 越大代表這次 action 越成功。假設這次選 Silent 而且 reward 不錯，那就增加 `p_C`；假設這次選 Betray 而且 reward 不錯，那就降低 `p_C`，因為 `p_C` 代表的是合作機率。

我這邊選的是 L_RP，所以 reward 好的時候會 reward selected action，reward 不好的時候也會有 penalty 去修正機率。這和 L_RI 最大的差別是，L_RI 基本上只 reward、不太管 penalty。

`RLAgent.py`

RLAgent 我使用的是 DQN。直覺上它在學一張表，只是這張表不是手刻出來，而是用 neural network 近似：

```
Q(state, action) = 在這個 state 選這個 action 後，未來總 reward 大概會是多少
```

這題的 state 只有兩個數字：

```
[protagonist 上一步 action, opponent 上一步 action]
```

action 也只有兩個：

```
0 = Silent
1 = Betray
```

所以 DQN network 的輸入是 2 維，輸出是 2 維。輸出的兩個值分別代表「現在選 Silent 的預估價值」和「現在選 Betray 的預估價值」。

```
self.net = nn.Sequential(
    nn.Linear(obs_dim, 64),
    nn.ReLU(),
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, action_dim),
)
```

training 時我用 epsilon-greedy：

- 一開始 epsilon 很大，所以 agent 會多亂試，避免太早卡在某個 action。
- 後面 epsilon 慢慢下降，agent 會越來越相信自己學到的 Q value。
- evaluation 時就不亂試，直接選 Q value 最大的 action。

```
if explore and np.random.rand() < self.epsilon:
    return int(self.action_space.sample())

q_values = self.q_net(obs)
return int(torch.argmax(q_values, dim=1).item())
```

每次互動完，RLAgent 會把這次經驗放進 replay buffer：

```
(obs, action, reward, next_obs, done)
```

等資料夠多之後，每次隨機抽一批資料來訓練。這樣做的原因是，如果完全照時間順序學，資料彼此太像，DQN 會比較不穩。

DQN 的更新目標是：

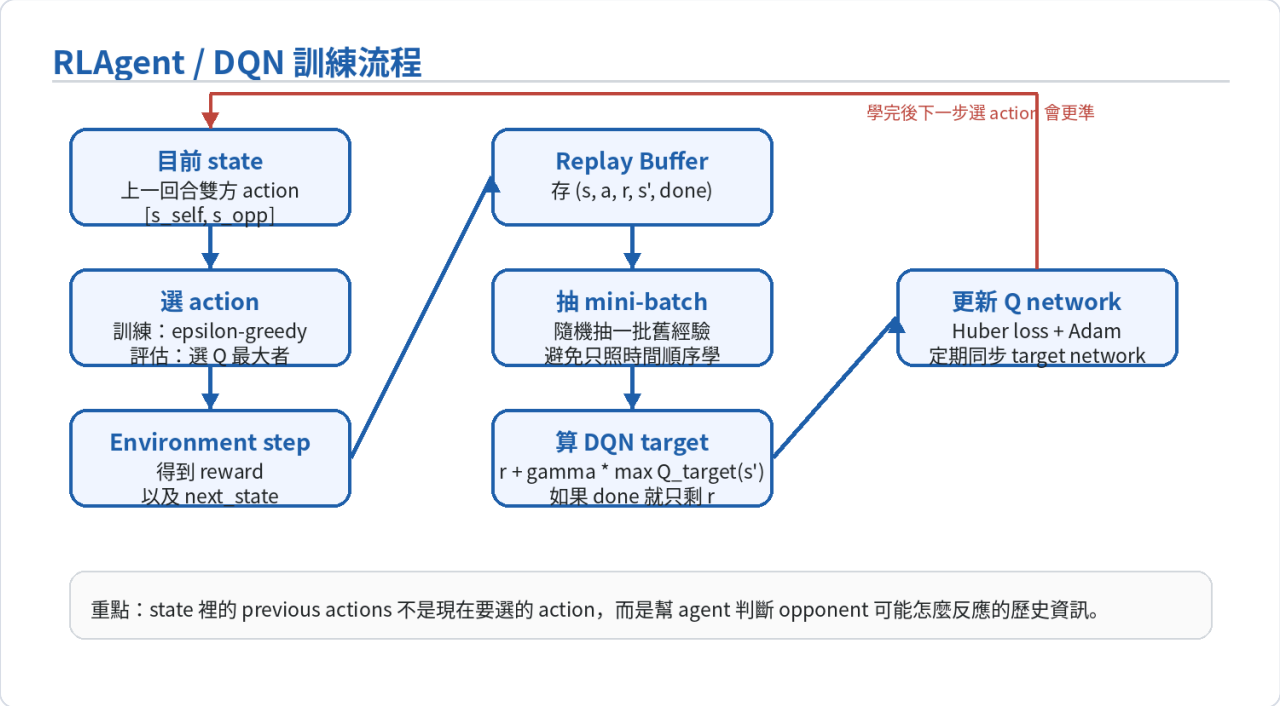
```
target = reward + gamma * max Q_target(next_state, next_action)
```

如果這一步已經是 episode 結束，後面就沒有 future reward，所以 target 只剩下當下 reward。

另外我有用 target network。簡單講就是準備兩個 network：

- `q_net`：真的在更新的 network。
- `target_q_net`：暫時固定住，用來算比較穩的 target。

每隔一段時間再把 `q_net` 的權重複製到 `target_q_net`，這樣訓練比較不會一直追著自己剛剛改掉的目標跑。

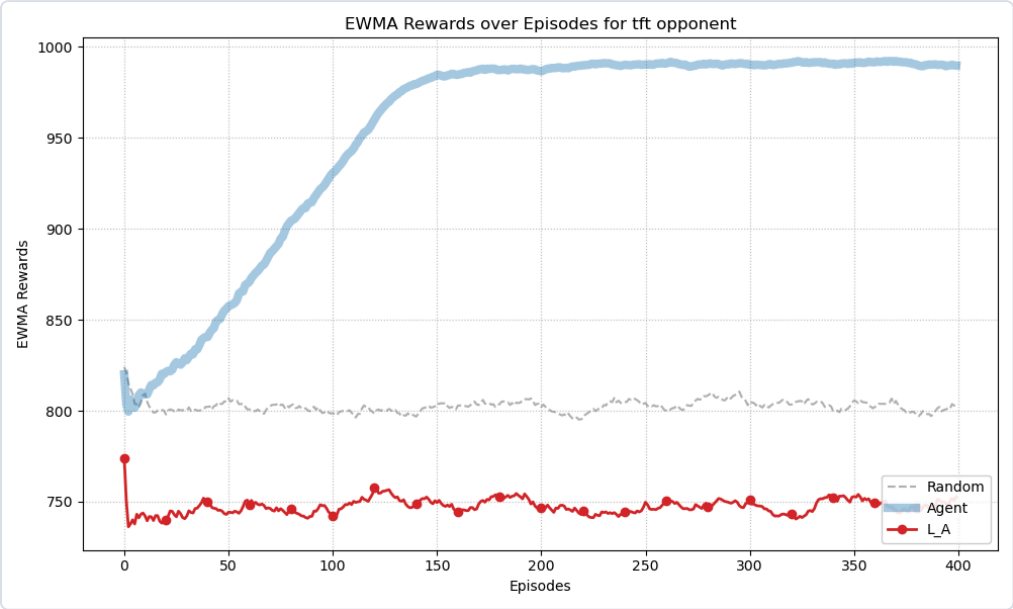


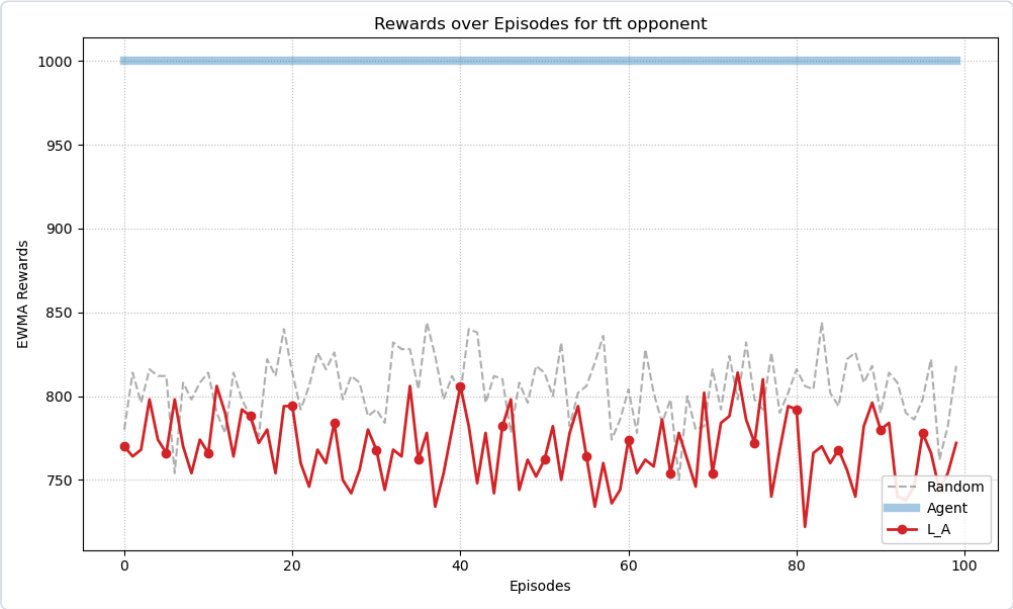
Result and Analysis

以下結果是用這個指令跑出來的：

```
conda run -n GameTheory python main.py
```

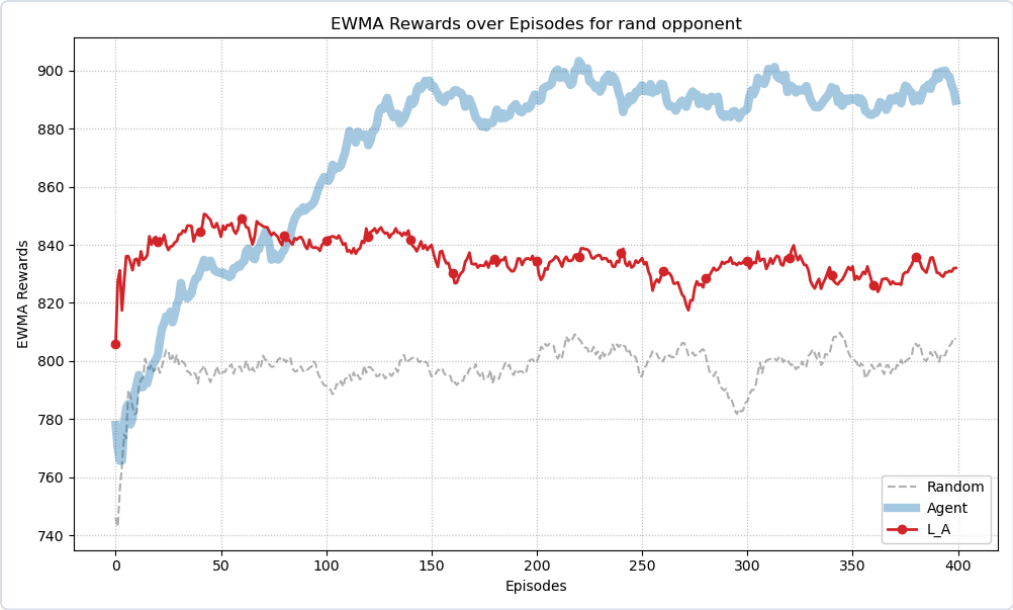
Tit for Tat opponent

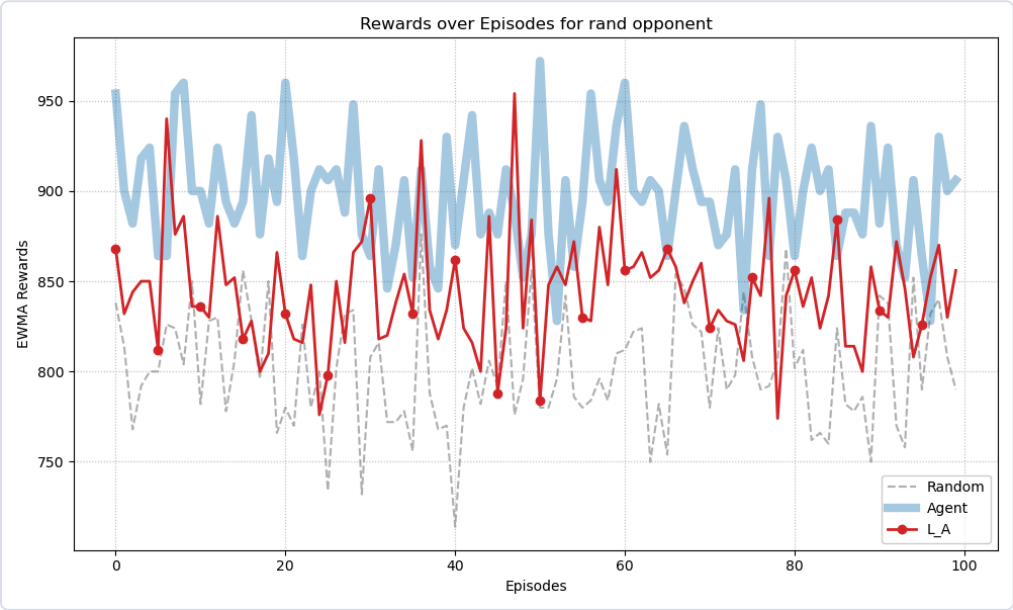




- TFT 的行為是第一步合作，後面模仿 protagonist 上一步。
- 這代表如果 protagonist 一直背叛，就會讓 opponent 後面也開始背叛。
- 從圖上看，RLAgent 最後可以學到很穩定的高 reward，evaluation 也幾乎固定在 1000 左右。
- LAAgent 因為只調一個整體合作機率，沒有辦法針對「上一回合發生什麼」做不同反應，所以在 TFT 這種看歷史的 opponent 上表現比較吃虧。

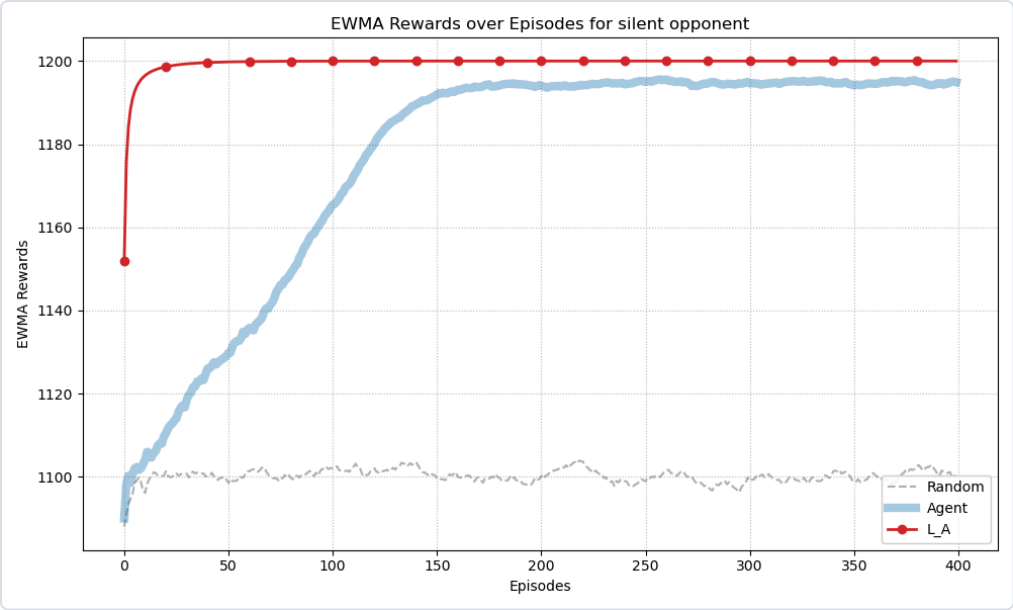
Random opponent

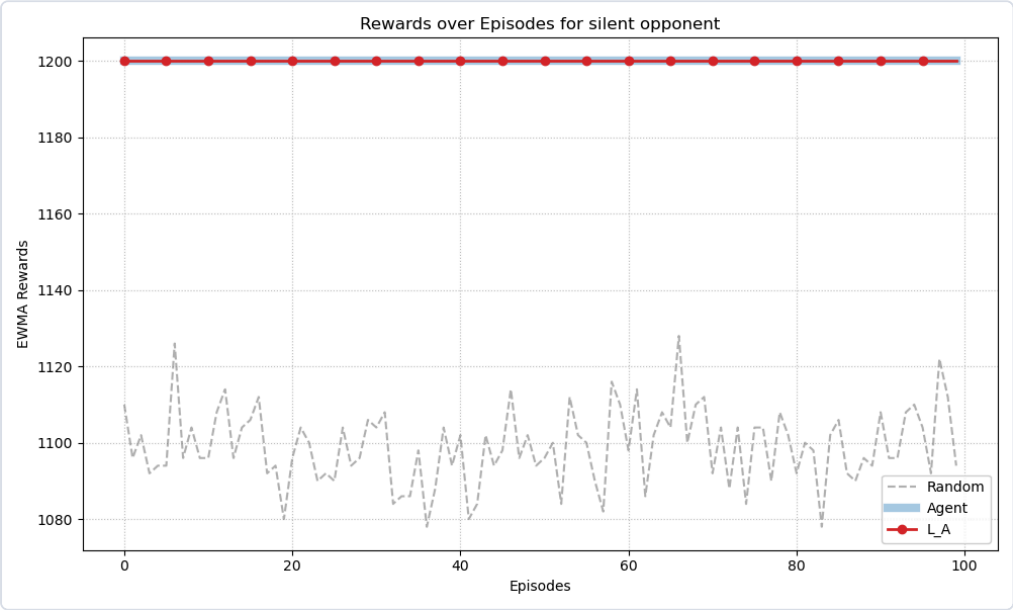




- Random opponent 不看歷史，所以 state 其實幫助比較小。
- 對 Random 來說，Betray 的期望 payoff 比 Silent 好，所以學到偏向 Betray 大致上會比較賺。
- 但因為 opponent 本身是隨機的，所以 evaluation reward 本來就會上下跳，不會像 TFT 那麼平滑。
- RLAgent 整體還是比 Random baseline 好，LAAgent 也有比 baseline 好一些，但波動比較明顯。

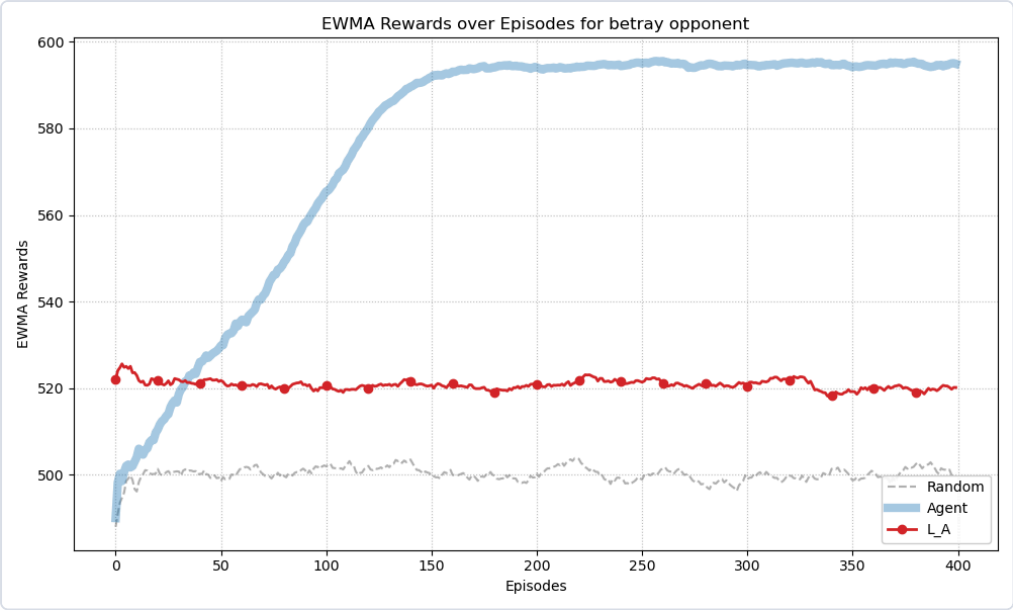
Always Silent opponent

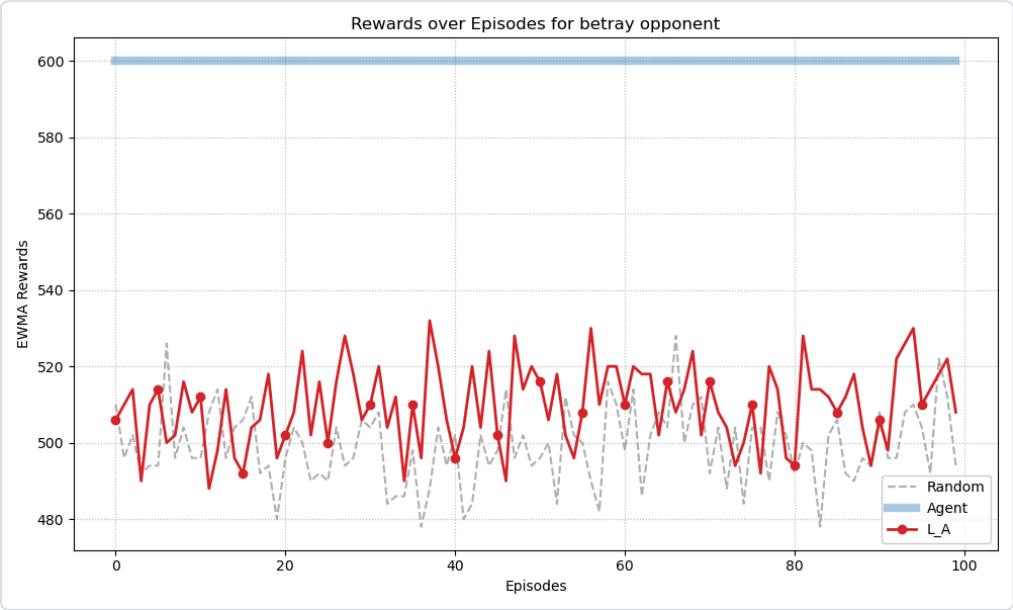




- 這個 opponent 永遠合作，所以 protagonist 最好的單步選擇就是 Betray，因為可以拿 12 分。
- 如果 100 回合都 Betray，理論上 episode reward 可以到 1200。
- 從圖來看，RLAgent 很快就學到接近滿分的策略。
- LAAgent 也會慢慢把機率推向 Betray，所以 evaluation 也會比 Random 好。

Always Betrayal opponent





- opponent 永遠背叛時，protagonist 選 Silent 只能拿 4，選 Betray 可以拿 6。
- 所以最佳反應仍然是 Betray，只是最高 episode reward 變成 600，不會像 Always Silent 那麼高。
- RLAgent 會學到穩定選 Betray，因此 reward 會接近 600。
- LAAgent 同樣可以學到偏向 Betray，但因為它是用機率慢慢調整，所以 evaluation 會比 RLAgent 更不穩一點。

Free Response Questions

Q1. Do you observe any interesting phenomena when your agents play this game?

我覺得最有趣的是，這題看起來都是 Prisoner's Dilemma，但其實對不同 opponent 最好的行為不完全一樣。

如果 opponent 是 Always Silent、Always Betrayal 或 Random，那 protagonist 幾乎只要學到「Betray 比 Silent 好」就可以拿到不錯的分數，所以 LA 這種沒有 state 的方法也可以有一定效果。

但如果 opponent 是 Tit for Tat，事情就變得比較像 repeated game。你這一步背叛雖然可能多拿一點，但會影響 opponent 下一步怎麼回應。這時候 RLAgent 因為可以看 state，也就是上一回合雙方做了什麼，所以比較有機會學到穩定策略。

Q2. Do you adopt any tricks in the agent, or just the vanilla algorithm?

LAAgent 基本上是照 Learning Automata 的 continuous L_RP 公式去寫，沒有特別硬刻 opponent strategy。

RLAgent 則是 DQN，但不是只寫最原始的 Q-learning table，因為我想讓它比較像一般 RL 實作，所以有加幾個 DQN 常見的穩定技巧：

- replay buffer：把過去經驗存起來，隨機抽樣訓練。
- target network：讓 target 不要每一步都跟著 online network 劇烈變動。
- epsilon-greedy：前期探索、後期利用學到的 Q value。
- Huber loss：比單純 MSE 對極端誤差穩一點。

但我沒有把 payoff matrix 的最佳解直接硬寫進 agent，所以它還是透過跟 environment 互動拿 reward 來學。

Q3. Do you consider states are necessary for an agent to make decisions?

我覺得要看 opponent 類型。

如果 opponent 是 Always Silent、Always Betrayal、Random，state 其實不是很必要。因為這些 opponent 不會根據歷史改變行為，protagonist 只要學到整體上哪個 action 比較好就可以。

但如果 opponent 會根據歷史行動，例如 Tit for Tat，state 就很重要。因為 TFT 下一步會模仿 protagonist 上一步，所以 protagonist 如果不知道上一回合自己做了什麼，就很難判斷 opponent 接下來可能會怎麼做。

所以我的理解是：state 不一定永遠必要，但只要 opponent 或 environment 有記憶，state 就會變得很重要。

Q4. In MDP of standard Reinforcement Learning, "State" and "Action" are distinct concepts. However, in our Repeated Prisoner's Dilemma, the

observation state is explicitly defined as the previous actions. Why is it necessary to formulate previous actions as the current state?

這邊容易混淆的是：「上一回合的 action」雖然名字也叫 action，但對現在這一步來說，它已經不是我要選的 action，而是歷史資訊。

在這一回合，agent 真正要選的是新的 action：Silent 或 Betray。而 state 則是在描述「目前局面長什麼樣子」。在 repeated Prisoner's Dilemma 裡，目前局面最重要的資訊就是上一回合雙方做了什麼。

例如 TFT opponent 會看 protagonist 上一步，所以如果 state 裡沒有 previous actions，agent 看到的每一步都長得差不多，它就不知道 opponent 為什麼這一步會背叛或合作。

因此把 previous actions 放進 state，是為了讓這個問題比較接近 MDP。也就是說，在知道目前 state 和自己接下來 action 的情況下，agent 才比較能預測下一個 state 和 reward。

補充：關於 Random opponent 的曲線比較

我後來有注意到，`main.py` 是用同一個 environment 依序跑 RandomAgent、RLAgent、LAAgent。對 `rand` opponent 來說，這代表三個 agent 看到的 random opponent action 不是完全同一串，而是同一個 RNG 後面不同段的結果。

所以這種比較在長期平均上仍然合理，但不是最嚴格的 paired comparison。更嚴格的做法會是每個 agent 都重新建立 environment 或重新設定 seed，讓它們面對完全相同的 random sequence。

不過因為 `main.py` 是助教提供的主要入口，而且 PDF 也說 main logic 不要大改，所以我最後沒有修改這段，只是在這邊補充說明。

Verification

我最後有跑以下檢查：

```
conda run -n GameTheory python -m py_compile BaseAgent.py RandomAgent.py LAAgent.py RLAgent.py
PrisonersDilemmaEnv.py plot.py main.py
```

```
conda run -n GameTheory python -c "from gymnasium.utils.env_checker import check_env; from
PrisonersDilemmaEnv import PrisonersDilemmaEnv; check_env(PrisonersDilemmaEnv(opponent_type='tft'),
skip_render_check=True)"
```

```
conda run -n GameTheory python main.py
```

最後 `main.py` 有正常跑完四種 opponent，並且產生八張圖：

- `result_tft_train.png`
- `result_tft_eval.png`
- `result_rand_train.png`
- `result_rand_eval.png`
- `result_silent_train.png`
- `result_silent_eval.png`
- `result_betray_train.png`
- `result_betray_eval.png`